

VIBEGUARD: ASSEMBLY, SENSITIVITY TUNING & DASHBOARD HANDOFF GUIDE

Standard Operating Procedure (SOP) & Lab Handoff Manual | Version 1.0

Author: VibeGuard Systems Lead | Date: October 2026 | Target Audience: Assembly & Presentation Team | Status: Ready for Rig Assembly

HANDOFF OBJECTIVE & EXECUTIVE SUMMARY:

All core firmware (ESP32 SPI driver, ISO 10816 DSP) and software (WebSocket bridge, animated web dashboard) are 100% verified and operating. This document instructs the teammate handling physical acrylic assembly on **how to bolt down the rig properly**, **how to calibrate and log stepped unbalance data**, and **how to connect the hardware to the browser dashboard** for the viva demonstration.

PART 1: RIGID MECHANICAL ASSEMBLY PROCEDURE

CRITICAL ENGINEERING NOTE: WHY HAND-HOLDING SKEWS THE READINGS:

During preliminary testing, holding the motor and sensor stand with human fingers acted as a viscoelastic shock absorber. Human skin and muscle damp out high-frequency mechanical vibration, causing 2-bolt test readings to remain artificially low in the Yellow zone (~0.4g to 0.6g). Bolting the hardware firmly to the acrylic baseplate creates a **rigid mechanical transmission bridge** (Motor → Bracket → Acrylic → Stand → Sensor), producing sharp, repeatable, high-amplitude vibrations that reliably trigger the Red Alarm.

Step-by-Step Bolting Checklist:

- 1. N20 Motor Bracket:** Fasten the steel N20 bracket to the acrylic baseplate using two M3 screws and M3 nuts. Tighten firmly with a screwdriver and pliers so the bracket cannot flex or wobble.
- 2. ADXL345 Sensor Stand:** Bolt the 3D-printed rigid sensor bracket (FAB-02) onto the acrylic baseplate keeping exactly 20 mm parallel spacing from the motor face. Use M3 screws and nuts through the baseplate through-holes.
- 3. Sensor PCB Mounting:** Mount the blue ADXL345 PCB upright onto the vertical flange of the sensor stand using M3 standoffs/screws. Ensure the PCB is perpendicular to the baseplate and firmly fastened.
- 4. Rotor Arm Press-Fit:** Push the 3D-printed eccentric cam arm (FAB-01, 3.40 mm D-shaft) onto the N20 motor output shaft. Align the flat face with the motor D-flat. Push it until seated, leaving 1.0 mm clearance so it does not rub against the gearbox face.
- 5. Bench Stability:** Place the acrylic baseplate on a flat, sturdy table. Place 4 small rubber feet, foam pads, or a mousepad beneath the acrylic corners to prevent the baseplate from sliding while motor spins.

PART 2: FIRMWARE BENCHMARKING & SENSITIVITY TUNING

The firmware loaded on the ESP32 (VibeGuard_Benchmark_Logger.ino) contains an interactive calibration and logging engine that computes AC Vector RMS over 128-sample windows (800 Hz ODR) with DC gravity bias removal.

The 4-Stage Stepped Unbalance Protocol:

Open the Arduino IDE Serial Monitor (or terminal) at **115200 baud**. Execute the following sequential steps:

Command	Test Stage	Mass Configuration	Expected VRMS	ISO Severity & LED
Send '0'	Stage 0: Baseline	Bare arm, 0 bolts/nuts (0.0g)	0.080 - 0.120 g	Zone A (Good) • Solid Green
Send '1'	Stage 1: Mild Unbal.	1x M3x20 bolt alone (~2.5g)	0.220 - 0.340 g	Zone B (Acceptable) • Green
Send '2'	Stage 2: Mod. Unbal.	M3 bolt + 1 nut (~3.4g)	0.380 - 0.650 g	Zone C (Warning) • Solid Yellow
Send '3'	Stage 3: Severe Unbal.	M3 bolt + 2 nuts (~4.3g)	0.750 - 1.300 g+	Zone D (Alarm) • Solid Red
Send 's'	Generate Report	All stages completed	Full comparative ledger	Prints Markdown Table for Viva
Send 'c'	Reset Ledger	Clear all recorded stages	--	Resets baseline for new run

How to Tune Sensitivity Thresholds in Code (If Needed):

If you want the **Red Alarm LED** to trigger at a lower vibration level for your 2-nut test on acrylic, open VibeGuard_Benchmark_Logger.ino (lines 37-38) in Arduino IDE:

```
// DSP Window & ISO 10816 Thresholds in VibeGuard_Benchmark_Logger.ino
#define RMS_WARN_LIMIT  0.35f  // ISO Zone B/C Warning threshold (Yellow LED: >= 0.35g)
#define RMS_ALARM_LIMIT 0.70f  // ISO Zone C/D Alarm threshold   (Red LED:   >= 0.70g)

// TUNING TIP: If 2 nuts produce ~0.55g on your acrylic setup and you want Red immediately:
// Change RMS_ALARM_LIMIT to 0.50f or 0.55f, then re-upload with auto_flash_esp32.sh!
```

PART 3: CONNECTING TO THE LIVE WEB DASHBOARD

CRITICAL GOLDEN RULE & PORT CONFIGURATION:

YOU MUST CLOSE THE ARDUINO SERIAL MONITOR BEFORE STARTING THE DASHBOARD!

On Linux and Windows, a serial port (e.g. /dev/ttyUSB0 or COM5) can only be opened by **one single application at a time**. If the Arduino IDE Serial Monitor is open, the web server will fail with Resource busy (EBUSY). Close the monitor tab first!

1. Start the Web Dashboard Server:

Open a terminal on the laptop and execute the following commands:

```
# 1. Navigate to the dashboard directory:
cd ~/Documents/PROJECT_ORGANIZED/07_SEMESTER_EXECUTION/04_Firmware/VibeGuard_Web_Dashboard

# 2. Start the Node.js server with the ESP32 serial port:
SERIAL_PORT=/dev/ttyUSB0 npm start
# (Or use auto-detection: AUTO_SERIAL=true npm start)

# Windows PowerShell alternative:
# $env:SERIAL_PORT='COM5'; npm start
```

2. Open the Browser Interface:

Open Google Chrome, Chromium, or Firefox to the following URL:

- **Live Hardware Mode:** <http://127.0.0.1:8765/vibeguard-dashboard.html?source=live>
- **Offline Preview Mode:** <http://127.0.0.1:8765/vibeguard-dashboard.html?source=preview>

3. Presentation & Operation Features:

- **Full-Screen Presentation Mode:** Press the '**P**' key on the keyboard to enter clean full-screen presentation mode for your viva presentation.
- **Pause Button:** Click the pause icon in the top header to freeze waveform curves for professor inspection while background acquisition continues.
- **Export Session CSV:** Click the blue '**Export session CSV**' button in the readings panel to download all logged timestamped data points as a .csv spreadsheet.
- **Animated Digital Twin:** The isometric acrylic rig SVG visualizer shakes and changes color in real-time according to measured vibration severity.

PART 4: HARDWARE WIRING & STATUS LED REFERENCE

Signal	ESP32 GPIO	ADXL345 / Component	Functional Description
CS	GPIO 21	CS Pin (Pin 7)	SPI Chip Select (Active LOW)
SCK	GPIO 18	SCL Pin (Pin 1)	1 MHz Hardware SPI Clock (Mode 2)
MISO	GPIO 19	SDO Pin (Pin 2)	Master In Slave Out (Sensor Data)
MOSI	GPIO 23	SDA Pin (Pin 3)	Master Out Slave In (Commands)
VCC / GND	3V3 / GND	VCC / GND	Regulated 3.3V Logic & Common Ground
LED Green	GPIO 25	RGB Green Pin	Normal Vibration (< 0.35g VRMS)
LED Blue	GPIO 26	RGB Blue Pin	Booting / 4-sec Calibration Average
LED Red	GPIO 27	RGB Red Pin	Critical Unbalance Alarm (>= 0.70g)

PART 5: RAPID TROUBLESHOOTING MATRIX

Observed Symptom	Likely Root Cause	Prescribed Corrective Action
Port busy (EBUSY) error when starting dashboard	Arduino IDE Serial Monitor is still running and locking /dev/ttyUSB0.	Close the Serial Monitor window in Arduino IDE. Run <code>ls /dev/ttyUSB0</code> to find and kill the locking PID.
LED stays Yellow during 2-bolt test instead of Red	Rig is held by hand, dampening vibration; or bolts are loose on acrylic.	Firmly tighten all M3 screws on acrylic plate. If vibration is $\sim 0.55g$, adjust <code>RMS_ALARM_LIMIT</code> to 0.50f in firmware.
Sensor flatlined / Blue search light	ADXL345 jumper wire disconnected or CS wire loose.	Verify DuPont wires: CS to GPIO 21, SCK to 18, MISO to 19, MOSI to 23. Check 3.3V rail voltage.
Garbled characters in terminal during motor start	Electrical brush noise (EMI) from 12V DC motor commutation.	Normal transient behavior. Clean ASCII resumes automatically within 200 ms.
Browser shows 'Bridge offline'	Node.js server process stopped or port 8765 blocked.	Run <code>SERIAL_PORT=/dev/ttyUSB0 npm start</code> in the dashboard folder. Check http://127.0.0.1:8765/health .